

DBMS Module 3: Relational Database Design

Relational Database Design

Relational database design is the process of structuring a database in such a way that it ensures data integrity, minimizes redundancy, and supports efficient data retrieval. The foundation of good database design is based on organizing data into well-defined **schemas** that follow specific design guidelines.

Design Guidelines for Relational Schemas

A **relational schema** defines the structure of a database in terms of tables (relations), columns (attributes), and the relationships between tables. Effective schema design follows these key principles:

1. **Minimize Redundancy:** Avoid storing the same data multiple times to prevent inconsistencies and unnecessary space usage.
 2. **Eliminate Update Anomalies:** Ensure that the schema allows for efficient updates, inserts, and deletes without causing data anomalies.
 3. **Use Meaningful Primary Keys:** Choose primary keys (unique identifiers) that are stable and meaningful within the context of the table.
 4. **Reduce Null Values:** Design the schema so that attributes that do not apply to some records do not result in excessive null values.
-

Functional Dependencies (FDs)

A **functional dependency** is a relationship between two attributes, typically in a database table. If an attribute **B** is functionally dependent on attribute **A**, then knowing the value of **A** allows us to uniquely determine the value of **B**.

- **Notation:** $A \rightarrow B$ (Attribute **A** determines Attribute **B**)

- **Example:** In a table of students, if the **Student_ID** determines the **Student_Name**, we can say that `Student_ID → Student_Name`.

FDs help identify redundancy in a table, which is critical for normalizing databases (the process of organizing tables to reduce redundancy).

Normalization

Normalization is the process of organizing a database into smaller, more manageable tables while ensuring data integrity and minimizing redundancy. This is achieved by applying a series of normal forms, which define different levels of database organization.

First Normal Form (1NF)

A relation is in **First Normal Form (1NF)** if it satisfies the following conditions:

1. **Atomicity:** All attribute values are atomic, meaning each value is indivisible. (No multi-valued or composite attributes.)
2. **Uniqueness:** There must be a primary key that uniquely identifies each record.

Example: A table of students with multiple phone numbers must split the phone numbers into separate records or columns to be in 1NF.

Second Normal Form (2NF)

A relation is in **Second Normal Form (2NF)** if:

1. It is in 1NF.
 2. **No Partial Dependencies:** No non-prime attribute (non-primary key attribute) is dependent on only part of a candidate key.
 - **Explanation:** If the primary key is composite (consists of multiple attributes), each non-key attribute must depend on the whole primary key, not just a part of it.
 - **Example:** In a table with `Student_ID` and `Course_ID` as the primary key, if `Student_Name` depends only on `Student_ID`, we need to move `Student_Name` to a different table to satisfy 2NF.
-

Third Normal Form (3NF)

A relation is in **Third Normal Form (3NF)** if:

1. It is in 2NF.
 2. **No Transitive Dependencies:** No non-prime attribute should depend on another non-prime attribute.
 - **Explanation:** Non-key attributes should not indirectly depend on the primary key through another non-key attribute.
 - **Example:** If $\text{Student_ID} \rightarrow \text{Department_ID} \rightarrow \text{Department_Name}$, we must remove the transitive dependency by creating a separate table for departments.
-

Decomposition Using Functional Dependencies

To achieve higher normal forms, we may need to **decompose** a relation into smaller relations that maintain the functional dependencies. This helps eliminate redundancy and anomalies while preserving the original data.

Boyce-Codd Normal Form (BCNF)

A relation is in **Boyce-Codd Normal Form (BCNF)** if:

1. It is in 3NF.
 2. **Every determinant is a candidate key:** For every functional dependency $A \rightarrow B$, attribute A must be a superkey.
 - **Explanation:** BCNF is a stricter form of 3NF, addressing cases where a relation is in 3NF but still has some anomalies.
 - **Example:** In a table where a non-candidate key attribute functionally determines a candidate key, the relation is not in BCNF and needs decomposition.
-

Decomposition Using Multivalued Dependencies (MVDs)

A **multivalued dependency** occurs when one attribute determines a set of values for another attribute independently of other attributes in the relation.

- **Notation:** $A \twoheadrightarrow B$ (A multivalued determines B)
 - **Decomposition:** A relation with multivalued dependencies can be decomposed into two or more relations without losing information.
-

Fourth Normal Form (4NF)

A relation is in **Fourth Normal Form (4NF)** if:

1. It is in BCNF.
 2. **No multivalued dependencies** exist in the relation.
 - **Explanation:** 4NF ensures that multivalued dependencies are properly handled by decomposing the relation into smaller relations.
 - **Example:** In a table where `Student_ID` determines multiple `Phone_Numbers` and `Addresses`, we decompose it into separate tables for phone numbers and addresses to satisfy 4NF.
-

Database Design and Implementation Process

The database design and implementation process involves several stages to ensure that the database meets all user requirements while being efficient and easy to manage.

1. Requirements Analysis

- Identify what data needs to be stored, how the data will be used, and the relationships between different pieces of data.

2. Conceptual Design

- Create a high-level conceptual schema, usually represented as an **Entity-Relationship (ER) diagram**, which models the entities and relationships in the database.

3. Logical Design

- Convert the conceptual schema into a **logical schema**, which maps entities and relationships to tables and columns in a relational database model.

4. Physical Design

- Define the **physical storage structure** of the database, including indexing, partitioning, and optimization strategies for performance.

5. Implementation

- Implement the schema in the database management system (DBMS), load data, and ensure it satisfies the functional and performance requirements.

6. Testing and Evaluation

- Test the database by running typical queries and checking for correctness, efficiency, and scalability.

7. Maintenance

- Continuously monitor and optimize the database to handle changes in data volume, query patterns, and system requirements.

Summary

1. **Design Guidelines:** Minimize redundancy and avoid update anomalies by using meaningful keys and proper design principles.
2. **Normalization:** Organize tables through normal forms (1NF, 2NF, 3NF, BCNF, 4NF) to reduce redundancy and maintain data integrity.
3. **Functional Dependencies:** Use functional dependencies to detect relationships between attributes and guide normalization.
4. **Decomposition:** Decompose tables based on functional and multivalued dependencies to achieve higher normal forms.
5. **Database Design Process:** Follow a structured process from requirements gathering to implementation to ensure a well-optimized, reliable database.